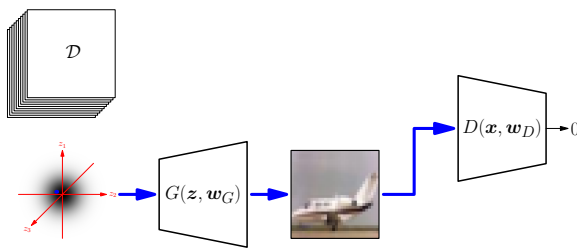


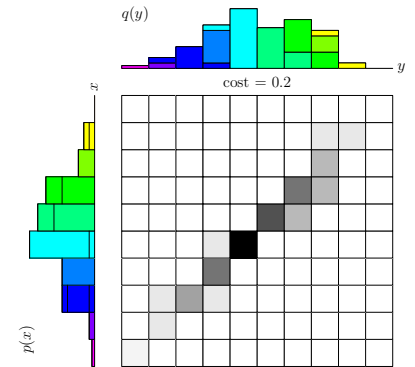
Wasserstein GANs



GANs, Wasserstein distance, Duality, WGANs

Outline

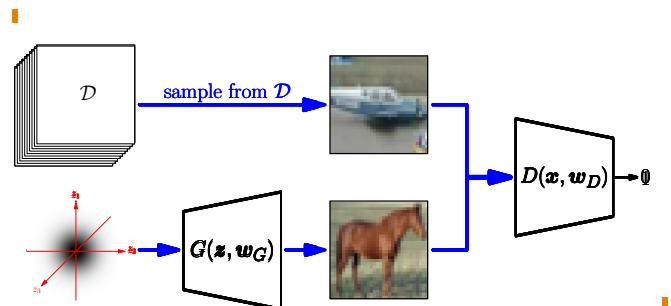
1. GANs
2. Wasserstein Distance
3. Wasserstein GANs



Generative Adversarial Networks

- One of the applications of Deep Learning that has most excited the public are **Generative Adversarial Networks** or **GANs**
- Their aim is to generate new random samples from the same distribution as some training set, $\mathcal{D}$
- Their number of real world applications are questionable
- But nobody cares because they are cool
- *Out of date warning: someone invented diffusion models*

How GANs Work



Training GANs

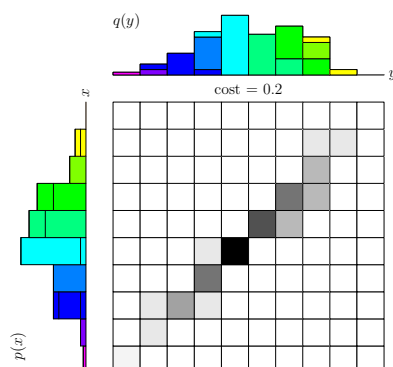
- The loss of the generator depends on its ability to trick the discriminator
- The loss of the discriminator depends on its ability not to be tricked
- We try to train the two networks simultaneously
- We hope that over time the generator produces better and better fakes

Problems of GANs

- GANs are notoriously difficult to train
- The generator and discriminator training can decouple
- Often the discriminator becomes too good at correctly identifying the generated images
- Then there can be little gradient information to help the generator as every small change in parameters doesn't significantly change the discriminator decision
- To try to solve this problem we first make a seemingly unconnected diversion

Outline

1. GANs
2. Wasserstein Distance
3. Wasserstein GANs



Measuring Distances Between Distributions

- In many machine learning tasks we want to minimise the distance between two probability distributions
- This requires that we can measure distances between probability distributions
- One prominent measure is the Kullback-Leibler or KL divergence

$$\text{KL}(p||q) = \int p(x) \log \left(\frac{p(x)}{q(y)} \right) dx$$

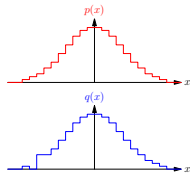
- This is very commonly used in ML (e.g. VAEs, Variational Approximation)

Trouble with KL

- KL-divergences are non-negative quantities that are minimised when the two probability distributions are the same
- They are not distances (they aren't symmetric and they don't satisfy the triangular inequality)

We don't really care about this, but what

- we do care about is that if $q(x) = 0$ when $p(x) \neq 0$ then $\log\left(\frac{p(x)}{q(y)}\right)$ diverges

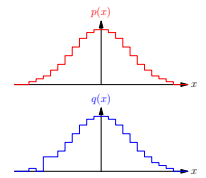


- We can therefore have distributions that seem very similar but their KL-divergence is huge (or infinite)

Wasserstein Distance

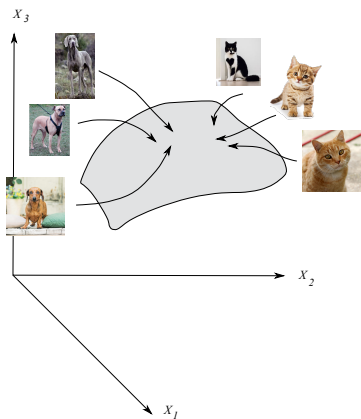
- A more benign measure of the differences between two probability functions is the **Wasserstein** or **Earth Moving** distance

This is a true distance, but more importantly, it measures distance in a very natural way so that distributions that are close has a small Wasserstein distance



- Although this seems contrived if our probability distribution represents the probability of a 128×128 matrix of real valued triples represents an image of dog, then it is easy to imagine that the Wasserstein distance may be more benign than the KL-divergence

High Probability Manifold



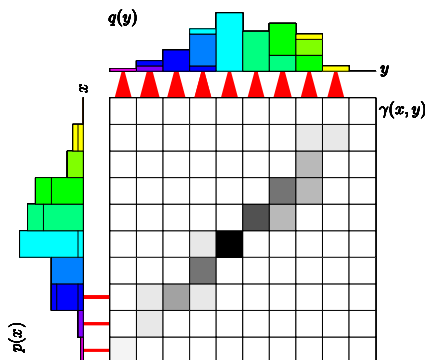
Transportation Policy

- But how do we formalise the Wasserstein distance?
- A good place to start is to define a transportation policy $\gamma(x, y)$ with

$$\int \gamma(x, y) dy = p(x) \quad \int \gamma(x, y) dx = q(y)$$

- This looks like a joint probability distribution, but we interpret $\gamma(x, y)$ as the amount of probability mass/density that we transfer from $p(x)$ to $q(y)$

Transportation Policy



The Cost of Transport

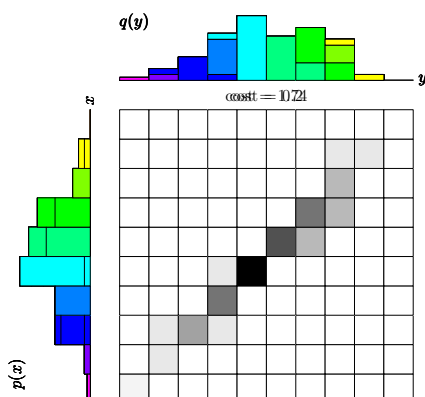
- We want to choose the transportation policy that minimises the amount of probability mass we need to move
- Let $d(x, y) = \|x - y\|$ be a distance measure then the cost of a transportation policy is

$$C(\gamma) = \int \int d(x, y) \gamma(x, y) dx dy = \mathbb{E}_{\gamma}[d(x, y)]$$

where we interpret $\gamma(x, y)$ as a probability distribution

- Usually we take $d(x, y)$ to be the Euclidean distance, but we can choose any distance

Transportation Cost



The Wasserstein Distance

- The Wasserstein distance $W(p, q)$ between two probability distributions is defined as

$$W(p, q) = \min_{\gamma \in \Lambda(p, q)} \mathbb{E}_{\gamma}[d(x, y)]$$

- Where $\Lambda(p, q)$ is the set of joint distributions $\gamma(x, y)$ such that

$$\int \gamma(x, y) dy = p(x) \quad \int \gamma(x, y) dx = q(y)$$

- To compute the Wasserstein distance we have to solve a minimisation task!
- This looks nasty, but it is a (continuous) linear programming problem!
- Suppose p and q were discrete distribution (i.e. x and y only take discrete points)
- Then we could treat each value of $\gamma(x, y)$ as an element of a vector γ and each value of $d(x, y)$ as an element of a vector D
- Our objective is to choose γ to minimise $D^T \gamma$

$$\sum_j \gamma(x_i, y_j) = p(x_i) \quad \sum_i \gamma(x_i, y_j) = q(y_j)$$

$$A\gamma = P$$

$$\begin{pmatrix} \gamma(x_1, y_1) \\ \gamma(x_2, y_1) \\ \vdots \\ \gamma(x_n, y_1) \\ \gamma(x_1, y_2) \\ \gamma(x_2, y_2) \\ \vdots \\ \gamma(x_n, y_2) \\ \vdots \\ \gamma(x_1, y_n) \\ \gamma(x_2, y_n) \\ \vdots \\ \gamma(x_n, y_n) \end{pmatrix} = \begin{pmatrix} q(y_1) \\ q(y_2) \\ \vdots \\ q(y_n) \\ p(x_1) \\ p(x_2) \\ \vdots \\ p(x_n) \end{pmatrix}$$

Lagrange Formulation

- For discrete distributions

$$\min_{\gamma} D^T \gamma$$

subject to $A\gamma = P, \quad \gamma \geq 0$

- Writing the Lagrangian

$$\mathcal{L}(\gamma, \alpha) = D^T \gamma - \alpha^T (A^T \gamma - P)$$

where α is a vector of Lagrange multipliers

- The solution to the discrete optimisation problem is given by

$$\min_{\gamma} \max_{\alpha} \mathcal{L}(\gamma, \alpha)$$

Explicit Form

- We can write a Lagrangian for the original problem

$$\mathcal{L} = \sum_{i,j} d(x_i, y_j) \gamma(x_i, y_j) - \sum_i \alpha(x_i) \left(\sum_j \gamma(x_i, y_j) - p(x_i) \right) - \sum_j \beta(y_j) \left(\sum_i \gamma(x_i, y_j) - q(y_j) \right)$$

subject to $\gamma(x_i, y_j) \geq 0$ where $\alpha(x_i)$ and $\beta(y_j)$ are Lagrange multipliers (they are components of α)

- Rearranging

$$\mathcal{L} = \sum_i \alpha(x_i) p(x_i) + \sum_j \beta(y_j) q(y_j) - \sum_{i,j} \gamma(x_i, y_j) (\alpha(x_i) + \beta(y_j) - d(x_i, y_j))$$

- This is equivalent to maximising $\sum_i \alpha(x_i) p(x_i) + \sum_j \beta(y_j) q(y_j)$, subject to

$$\forall i, j \quad \alpha(x_i) + \beta(y_j) \leq d(x_i, y_j)$$

Dual Form Constraint

- We note that $\alpha(x) + \beta(y) \leq d(x, y)$ for all x and y
- This has to be true when $x = y$ so that

$$\alpha(x) + \beta(x) \leq d(x, x) = 0$$

- So $\beta(x) = -\alpha(x) - \epsilon(x)$ where $\epsilon(x) \geq 0$
- But want to maximise

$$\int \alpha(x) p(x) dx + \int \beta(y) q(y) dy = \int \alpha(x) (p(x) - q(x)) dx - \int q(x) \epsilon(x) dx$$

- This is maximised when $\epsilon(x) = 0$ i.e. $\beta(x) = -\alpha(x)$

Dual Form

- We can rearrange

$$\mathcal{L}(\gamma, \alpha) = D^T \gamma - \alpha^T (A\gamma - P)$$

$$= P^T \alpha - \gamma^T (A^T \alpha - D)$$

- We note that $\gamma \geq 0$ so the dual problem is to find a vector α that maximises $P^T \alpha$ subject to the constraints $A^T \alpha \leq D$
- Although the vector form allows us to make connections with our earlier discussion of linear programming, it is a little difficult to interpret

Continuous Form

- We can write a Lagrangian for the continuous problem

$$\mathcal{L} = \iint d(x, y) \gamma(x, y) dx dy - \int \alpha(x) \left(\int \gamma(x, y) dy - p(x) \right) dx - \int \beta(y) \left(\int \gamma(x, y) dx - q(y) \right) dy$$

subject to $\gamma(x, y) \geq 0$ where $\alpha(x)$ and $\beta(y)$ are Lagrange multiplier functions

- Rearranging

$$\mathcal{L} = \int \alpha(x) p(x) dx + \int \beta(y) q(y) dy - \iint \gamma(x, y) (\alpha(x) + \beta(y) - d(x, y)) dx dy$$

- This is equivalent to maximising $\int \alpha(x) p(x) dx + \int \beta(y) q(y) dy$, subject to

$$\alpha(x) + \beta(y) \leq d(x, y)$$

Dual Form

- Thus the dual problem is to find a function $\alpha(x)$ —or a vector of functions $(\alpha(x_i))_i$ —that maximises

$$\int \alpha(x) (p(x) - q(x)) dx$$

- Subject to the constraint

$$\alpha(x) - \alpha(y) \leq d(x, y) = \|x - y\|$$

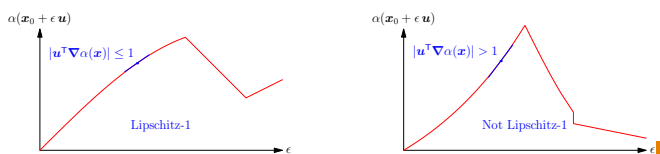
- This is a continuity constraint on the Lagrange multiplier function $\alpha(x)$ known as Lipschitz-1

Lipschitz-1 Functions

- We note for a Lipschitz-1 function and any unit vector u

$$u^T \nabla \alpha(x) = \lim_{\epsilon \rightarrow 0} \frac{\alpha(x + \epsilon u) - \alpha(x)}{\epsilon} \leq \frac{\epsilon \|u\|}{\epsilon} = \frac{\epsilon}{\epsilon} = 1$$

- That is, at every point the gradient in all directions must be less than 1 (since the gradient defines the direction of greatest increase it is both necessary and sufficient for $\|\nabla \alpha(x)\| \leq 1$ everywhere)



Calculating the Wasserstein Distance

- To recall the big picture we want to compute the Wasserstein distance

$$W(p, q) = \min_{\gamma \in \Lambda(p, q)} \mathbb{E}_{\gamma} [d(x, y)]$$

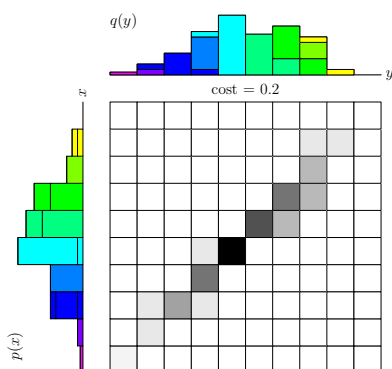
- For high dimensional objects $\gamma(x, y)$ would be a huge object to approximate
- Instead we can compute the Wasserstein distance in the dual formulation

$$W(p, q) = \max_{\alpha(x)} \int \alpha(x) (p(x) - q(x)) dx = \max_{\alpha} \mathbb{E}_p[\alpha(X)] - \mathbb{E}_q[\alpha(X)]$$

subject to the constraint that $\alpha(x)$ is a Lipschitz-1 function

Outline

1. GANs
2. Wasserstein Distance
3. Wasserstein GANs



Back to GANs

- What has this got to do with GANs?
- Suppose we want to minimise the distance between the distribution $p(x)$ of real images (of which $\mathcal{D}$ are samples) and the distribution $q(x)$ of images drawn from a generator
- We can use a normal GAN generator, $G(z, w_G)$, that generates an image when given a random variable $z \sim \mathcal{N}(0, I)$
- To do this we choose the weights, w_G of the generator to minimise

$$W(p, q) = \max_{\alpha(x)} (\mathbb{E}_{x \sim p}[\alpha(x)] - \mathbb{E}_{x \sim q}[\alpha(x)])$$

Estimating Expectations

- Although we can't compute $\mathbb{E}_p[\alpha(x)]$ and $\mathbb{E}_q[\alpha(x)]$ exactly, we can estimate them from samples

$$\mathbb{E}_p[\alpha(x)] \approx \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} \alpha(x), \quad \mathbb{E}_q[\alpha(x)] \approx \frac{1}{n} \sum_{i=1}^n \alpha(G(z_i, w_G))$$

- where $\mathcal{B} \subset \mathcal{D}$ is a minibatch of true images and $z_i \sim \mathcal{N}(0, I)$
- From this we can choose w_G to minimise

$$C = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} \alpha(x) - \frac{1}{n} \sum_{i=1}^n \alpha(G(z_i, w_G))$$

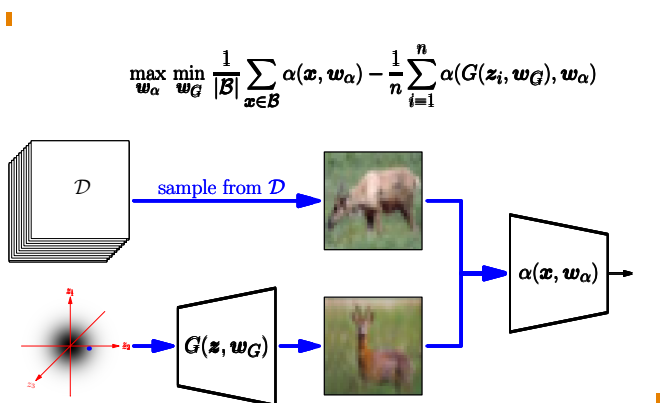
The Critic

- For this quantity to approximate the Wasserstein distance we need to find a function $\alpha(x, w_\alpha)$ that maximises C
- To do this we learn a second network, the critic or discriminator whose job it is to maximise

$$C = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} \alpha(x, w_\alpha) - \frac{1}{n} \sum_{i=1}^n \alpha(G(z_i, w_G), w_\alpha)$$

- The network $\alpha(x, w_\alpha)$ should be Lipschitz-1 (which we usually botched by, for example, by setting the spectral norm of the convolutional weight matrix to 1)

Wasserstein GANs



Lesson

- Wasserstein GANs are, at least for me, one of the most elegant pieces of theory in recent years
- By trying to minimise the Wasserstein distance between the distribution of a generator and a true distribution we arrive at optimising two adversarial networks just like a GAN
- This uses a rather beautiful dual formulation
- It is claimed that W-GANs solve many of the problems of traditional GANs